R-Type examples (opcode = `0110011`)

Format:
`funct7 (7) | rs2 (5) | rs1 (5) | funct3 (3) | rd (5) | opcode (7)`

1) `add x5, x6, x7`

- rd = x5 → `5` → `00101`
- rs1 = x6 → `6` → `00110`
- rs2 = x7 → `7` → `00111`
- funct3 = `000`
- funct7 = `0000000`
- opcode = `0110011` (0x33)

Binary fields:

funct7 = 0000000
rs2 = 00111
rs1 = 00110
funct3 = 000
rd = 00101
opcode = 0110011

Full 32-bit binary:

0000000 00111 00110 000 00101 0110011
00000000011100110000001010110011

Hex: `0x007302b3`

2) `sub x1, x2, x3`

- rd = x1 → `00001`
- rs1 = x2 → `00010`
- rs2 = x3 → `00011`
- funct3 = `000`
- funct7 = `0100000` (sub uses funct7=0100000)
- opcode = `0110011`

Binary fields:

funct7 = 0100000
rs2 = 00011
rs1 = 00010
funct3 = 000
rd = 00001
opcode = 0110011

Full 32-bit binary:

0100000 00011 00010 000 00001 0110011
01000000001100010000000010110011

Hex: `0x403100b3`

3) `and x10, x11, x12`

- rd = x10 → `01010`
- rs1 = x11 → `01011`
- rs2 = x12 → `01100`
- funct3 = `111` (AND)
- funct7 = `0000000`
- opcode = `0110011`

Binary fields:

funct7 = 0000000
rs2 = 01100
rs1 = 01011
funct3 = 111
rd = 01010
opcode = 0110011

Full 32-bit binary:

0000000 01100 01011 111 01010 0110011
00000000110001011111010100110011

Hex: `0x00c5f533`

S-Type examples (stores) (opcode = `0100011`)

S-type format:
`imm[11:5] (7)|rs2 (5)| rs1 (5) | funct3 (3) | imm[4:0] (5) | opcode(7)`

Immediate is split between bits [31:25] and [11:7]

1) `sw x5, 8(x6)` store word

- mnemonic: `sw x5, 8(x6)`
- rs1 = x6 → base register = `6` (`00110`)
- rs2 = x5 → source register = `5` (`00101`)
- imm = `8` → 12-bit signed imm = `000000001000`
    - imm[11:5] = `0000000` (bits 11..5)
    - imm[4:0] = `01000` (bits 4..0)

- funct3 = `010` (sw)
- opcode = `0100011`

Binary fields:

imm[11:5]=0000000
rs2 =00101
rs1 =00110
funct3 =010
imm[4:0] =01000
opcode =0100011

Full 32-bit binary:

0000000 00101 00110 010 01000 0100011
00000000010100110010010000100011

Hex: `0x00532423`

2) `sb x1, -4(x2)` store byte with negative immediate

- `sb x1, -4(x2)`
- rs1 = x2 → `00010`
- rs2 = x1 → `00001`
- imm = `-4` → 12-bit two's complement: `111111111100`
    - imm[11:5] = `1111111`
    - imm[4:0] = `11100`
- funct3 = `000` (sb)
- opcode = `0100011`

Binary fields:

imm[11:5]=1111111
rs2 =00001
rs1 =00010
funct3 =000
imm[4:0] =11100
opcode =0100011

Full 32-bit binary:

1111111 00001 00010 000 11100 0100011
11111110000100010000111000100011

Hex: `0xfe110e23`

3) `sh x10, 16(x11)` store halfword

- `sh x10, 16(x11)`
- rs1 = x11 → `01011`
- rs2 = x10 → `01010`
- imm = `16` → `000000010000`
    - imm[11:5] = `0000001`
    - imm[4:0] = `00000`
- funct3 = `001` (sh)
- opcode = `0100011`

Binary fields:

imm[11:5]=0000001
rs2 =01010

rs1 =01011
funct3 =001
imm[4:0] =00000
opcode =0100011

Full 32-bit binary:

0000001 01010 01011 001 00000 0100011
00000000101001011001100000100011

Hex: `0x00a59823`

I-Type ALU examples (opcode = `0010011`)

I-type format:
`imm[11:0] (12) | rs1 (5) | funct3 (3) | rd (5) | opcode (7)`
Immediate is sign-extended for arithmetic/logical immediates.

Shift-immediates are encoded in `imm` but use `imm[4:0]` as `shamt` and `imm[11:5]` to distinguish `srli`/`srai`.

1) `addi x5, x6, -4`

- rd = x5 → `00101`
- rs1 = x6 → `00110`
- imm = -4 → 12-bit two's complement `111111111100`
- funct3 = `000` (addi)
- opcode = `0010011`

Binary:

imm = 111111111100
rs1 = 00110
funct3 = 000
rd = 00101
opcode = 0010011

Full 32-bit binary:

111111111100 00110 000 00101 0010011
11111111110000110000001010010011

Hex: `0xffc30293`

2) `andi x1, x2, 15`

- `andi x1, x2, 15`
- rd = x1 → `00001`
- rs1 = x2 → `00010`
- imm = `15` → `000000001111`
- funct3 = `111` (andi)
- opcode = `0010011`

Binary:

imm = 000000001111
rs1 = 00010
funct3 = 111
rd   = 00001
opcode = 0010011

Full 32-bit binary:

000000001111 00010 111 00001 0010011
00000000111100010111000010010011

Hex: `0x00f17093`

3) `slli x10, x11, 3` (logical left immediate)

- `slli x10, x11, 3`
- rd = x10 → `01010`
- rs1 = x11 → `01011`
- shamt = 3 → `00011` (in imm[4:0])
- imm[11:5] = `0000000` (in RV32I)
- funct3 = `001` (slli)
- opcode = `0010011`

Binary:

imm[11:5] = 0000000
imm[4:0] = 00011
rs1 = 01011
funct3 = 001
rd = 01010
opcode = 0010011

Full 32-bit binary:

0000000 00011 01011 001 01010 0010011
00000000001101011001010100010011

Hex: `0x00359513`

B-Type (branch) examples (opcode = `1100011`)

B-type format:
`imm[12] | imm[10:5] (6) | rs2 (5) | rs1 (5) | funct3 (3) | imm[4:1] (4) | imm[11] | opcode (7)`
Effective branch target = PC + imm (imm is multiple of 2; least bit is zero). We show how imm splits into the fields.

> Important: In our encoding examples the `encode_b` uses the immediate value as a signed byte offset (e.g., 16, -8, 12). RISC-V assembler requires branch offsets to be multiples of 2; the immediate encoding reflects that.

1) `beq x1, x2, 16` (branch if equal, offset = 16 bytes)

- rs1 = x1 → `00001`
- rs2 = x2 → `00010`
- funct3 = `000` (beq)
- imm = 16 → binary 13-bit view `0000000010000` (we only need 12 effective bits but show split)
    - imm[12] = 0
    - imm[10:5] = `000000`

    - imm[4:1] = `1000` (bits 4..1)
    - imm[11] = 0

- opcode = `1100011`

Binary placement:

imm[12]=0
imm[10:5]=000000
rs2=00010
rs1=00001
funct3=000
imm[4:1]=1000
imm[11]=0
opcode=1100011

Full 32-bit binary:

0000000 0010 00001 000 1000 0 1100011
00000000001000001000100001100011

Hex: `0x00208863`

2) `bne x5, x6, -8` (branch if not equal, backward offset -8)

- rs1 = x5 → `00101`
- rs2 = x6 → `00110`
- funct3 = `001` (bne)
- imm = -8 → 12-bit two's complement branch immediate (represented in B fields):

    - Signed value `-8` → in 13-bit form used for mapping → binary `1111111111000` (we mask appropriately)

    - imm[12] = 1
    - imm[10:5] = `111110`
    - imm[4:1] = `1100`
    - imm[11] = 1

- opcode = `1100011`

Binary fields:

imm[12]=1

imm[10:5]=111110
rs2=00110
rs1=00101
funct3=001
imm[4:1]=1100
imm[11]=1
opcode=1100011

Full 32-bit binary:

1111111 00110 00101 001 1100 1 1100011
11111110011000101001110011100011

Hex: `0xfe629ce3`

(You can verify the offset: PC + (-8) encoded correctly.)

3) `blt x7, x8, 12` (branch if rs1 < rs2, signed compare)

- rs1 = x7 → `00111`
- rs2 = x8 → `01000`
- funct3 = `100` (blt)
- imm = 12 → binary splits:
    - imm[12]=0
    - imm[10:5]=`001100`
    - imm[4:1] = `1100`
    - imm[11] = 0
- opcode = `1100011`

Binary fields:

imm[12]=0
imm[10:5]=001100
rs2=01000
rs1=00111
funct3=100
imm[4:1]=1100
imm[11]=0
opcode=1100011

Full 32-bit binary:

0000000 01000 00111 100 1100 0 1100011
00000000100000111100011001100011

Hex: `0x0083c663`

J-Type examples

J-type (`jal`) format:
`imm[20] | imm[10:1](10) | imm[11] | imm[19:12] (8) | rd (5) |opcode(7)`
`jal` uses opcode `1101111` (0x6f).

1) `jal x1, 128` (jump and link)

- rd = x1 → `00001`
- imm = `128` (must be multiple of 2; here it's 128) → encode in J format:
    - imm (20-bit) = binary `00000010000000000000` (20 bits)
    - imm[20] = 0
    - imm[10:1] = `0000000000`
    - imm[11] = 0
    - imm[19:12] = `00000010`
- opcode = `1101111`

Binary fields:

imm[20]=0
imm[10:1]=0000000000
imm[11]=0
imm[19:12]=00000010
rd=00001
opcode=1101111

Full 32-bit binary:

0 00000010 0 0000000000 00001 1101111 (reordered to canonical)
00001000000000000000000011101111

Hex: `0x080000ef`

(That is `jal x1,128`.)

2) `jalr x1, 20(x2)` (jump and link register)

This is an I-type instruction using opcode `1100111` (0x67) and funct3 = `000`. It computes `t = rs1 + imm; PC = t & ~1; rd = return addr`.

- rd = x1 → `00001`
- rs1 = x2 → `00010`
- imm = 20 → 12-bit `000000010100`
- funct3 = `000`
- opcode = `1100111`

Binary:

imm = 000000010100
rs1 = 00010
funct3 = 000
rd = 00001
opcode = 1100111

Full 32-bit binary:

000000010100 00010 000 00001 1100111
00000001010000010000000011100111

Hex: `0x014100e7`